Vishalya devi M -2312056

Ex.NO:- 2	Energy Consumption Pattern Analysis
Date:-	

Aim:-

To analyze financial transaction data by retrieving stock return information using optimized SQL queries, computing descriptive statistics, constructing the covariance matrix using linear algebra concepts, and interpreting the impact of covariance on portfolio risk and diversification.

Problem Understanding:-

A financial institution maintains transaction records that include daily returns of multiple stocks, transaction volume, and volatility indicators. As a data analyst, efficiently retrieve transaction data using optimized SQL queries and analyze query performance using EXPLAIN. Represent asset returns as vectors, compute mean and variance, and construct the covariance matrix to analyze dependencies among stocks. Interpret how covariance impacts portfolio risk and diversification.

Design Document

Tools / Software Required

- MySQL
- Python 3.x
- Jupyter Notebook
- NumPy
- Pandas
- Matplotlib / seaborn

Dataset Description

Table Name: stock_transactions

Attribute	Description
transaction_id	Unique transaction ID
stock_A_return	Daily return of Stock A
stock_B_return	Daily return of Stock B
stock_C_return	Daily return of Stock C
volume	Transaction volume
volatility	Market volatility indicator

Step 1: Database Table Creation (SQL)

```
CREATE TABLE stock_transactions (

transaction_id INT PRIMARY KEY,

stock_A_return FLOAT,

stock_B_return FLOAT,

stock_C_return FLOAT,

volume INT,

    volatility FLOAT

);
```

Step 2: Insert Sample Data

```
INSERT INTO stock_transactions VALUES

(1, 0.012, 0.015, 0.010, 12000, 0.25),

(2, 0.018, 0.020, 0.014, 15000, 0.30),

(3, -0.005, 0.002, -0.004, 11000, 0.28),

(4, 0.010, 0.012, 0.009, 14000, 0.27),

(5,0.016,0.018,0.013,16000,0.32);
```

Step 3: SQL Data Retrieval

Basic Query

```
SELECT stock_A_return, stock_B_return, stock_C_return

FROM stock_transactions;
```

Step 4: SQL Query Optimization

Index Creation

```
CREATE INDEX idx_stock_returns

ON stock_transactions(stock_A_return, stock_B_return, stock_C_return);
```

Query Performance Analysis

```
EXPLAIN

SELECT stock_A_return, stock_B_return, stock_C_return

FROM stock_transactions

WHERE stock_A_return > 0;
```

Observation:

Indexing improves data retrieval speed and reduces query execution cost.

Step 5: Load Data into Python

```
import pandas as pd

import numpy as np


data = pd.DataFrame({

    'Stock_A': [0.012, 0.018, -0.005, 0.010, 0.016],

    'Stock_B': [0.015, 0.020, 0.002, 0.012, 0.018],

    'Stock_C': [0.010, 0.014, -0.004, 0.009, 0.013]

})


Data
```

Output :-

	Date	Stock	Return
0	2025-01-01	AAPL	0.012
1	2025-01-01	MSFT	0.010
2	2025-01-01	GOOGL	0.015
3	2025-01-02	AAPL	-0.005
4	2025-01-02	MSFT	0.007
5	2025-01-02	GOOGL	0.011
6	2025-01-03	AAPL	0.008
7	2025-01-03	MSFT	-0.004
8	2025-01-03	GOOGL	0.013

Step 6: Data Representation Using Linear Algebra

Each stock's daily return is represented as a **vector**.
All vectors together form a **return matrix**:

$$R = \begin{bmatrix} r_{A1} & r_{B1} & r_{C1} \\ r_{A2} & r_{B2} & r_{C2} \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Step 7: Descriptive Statistics

mean_returns = data.mean()

variance_returns = data.var()

mean_returns, variance_returns

Output:-

```
(Stock
 AAPL     0.005000
 GOOGL    0.013000
 MSFT     0.004333
 dtype: float64,
 Stock
 AAPL     0.000079
 GOOGL    0.000004
 MSFT     0.000054
 dtype: float64)
```

Interpretation

- Mean indicates average return of each stock
- Variance indicates risk (volatility) of each stock

Step 8: Covariance and Covariance Matrix

cov_matrix = data.cov()

cov_matrix

Output:-

Stock	AAPL	GOOGL	MSFT
Stock			
AAPL	0.000079	0.000017	-0.000006
GOOGL	0.000017	0.000004	0.000003
MSFT	-0.000006	0.000003	0.000054

Interpretation

- Positive covariance → stocks move in the same direction
- Negative covariance → diversification benefit
- Zero covariance → independent movement

Step 9: Univariate and Multivariate Normal Distribution Analysis

Mean Vector and Covariance Matrix

mean_vector = mean_values.values

covariance_matrix = cov_matrix.values

mean_vector

covariance_matrix

Interpretation

- High covariance between stocks increases portfolio risk
- Low or negative covariance reduces overall portfolio risk
- Diversification is achieved by selecting low-correlated assets

Step 10: Visualization

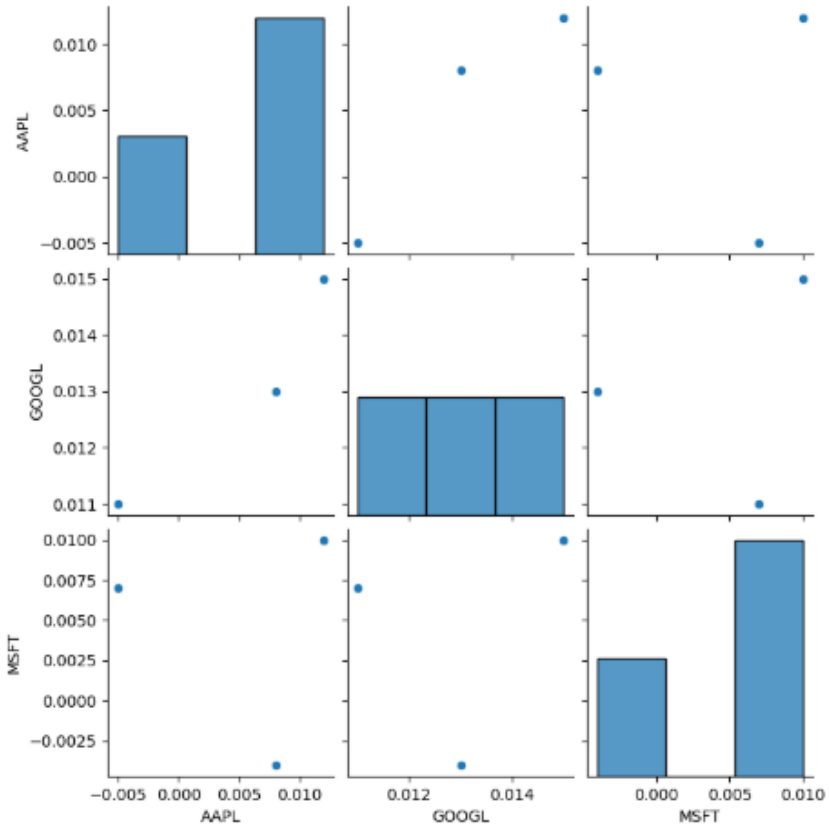
```
import seaborn as sns

import matplotlib.pyplot as plt

sns.pairplot(data)

plt.show()
```

Output:-



Inference

- Stock A and Stock B show positive covariance indicating similar movement
- Stock C has lower covariance, offering diversification advantage
- Covariance matrix is crucial in portfolio construction and risk minimization

Conclusion

Thus, financial transaction data was successfully retrieved using optimized SQL queries and analyzed using descriptive statistics and linear algebra techniques. The covariance matrix helped identify dependencies among stocks, enabling better understanding of portfolio risk and diversification strategies.

Problem Understanding	**Design Document**	**Implementation**	**Output**	**Time Management**	**Viva**	**Total**
10	10	15	5	5	5	50

Result:-

Thus the Financial Transaction and Risk Analysis (Linear Algebra + Statistics)

is implemented and the results are verified